

Parallel Algorithms



Matrix Multiplications – part 1

João L. Sobral

jls@di ...

Parallel Algorithms

Modern algorithm analysis/design:

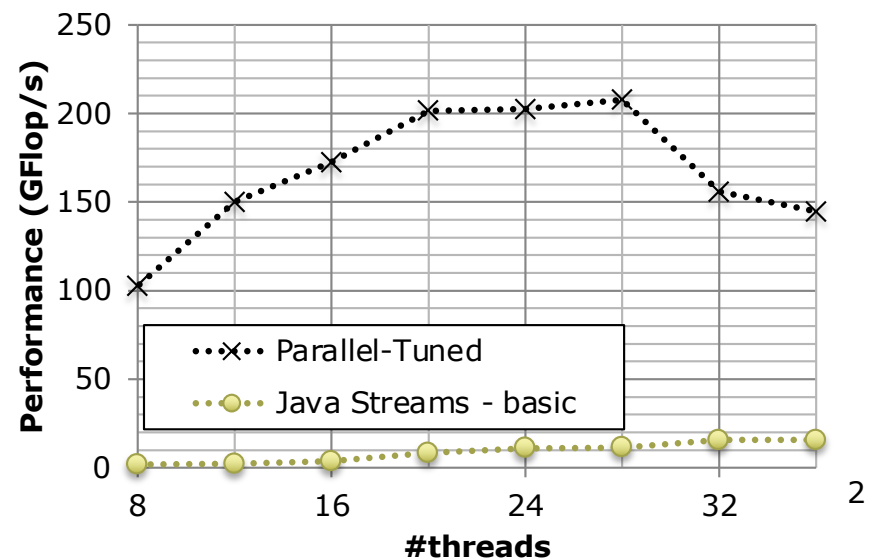
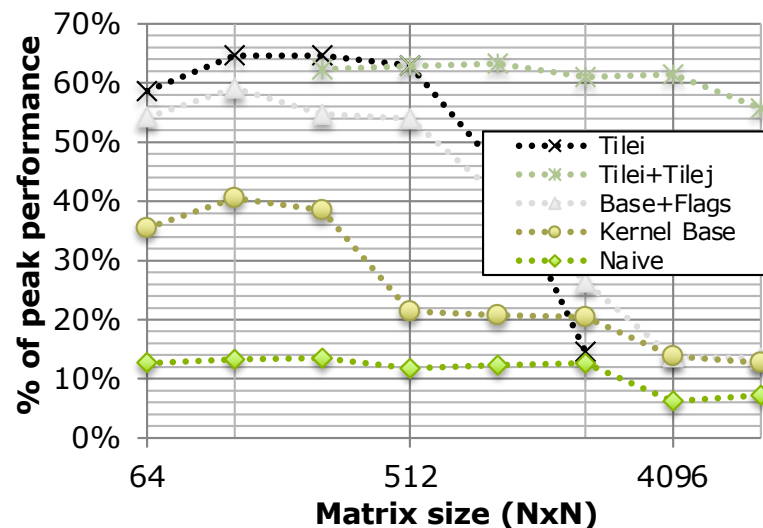
- Traditional complexity analysis: $O(x)$
 - Lower #instructions (the most **one** important issue)
- Data locality
- Vectorization
- Parallelization

← Both require data locality & parallelism!

Today's case study: matrix multiplication (Java, r641)

- Seq. Base VS optimized

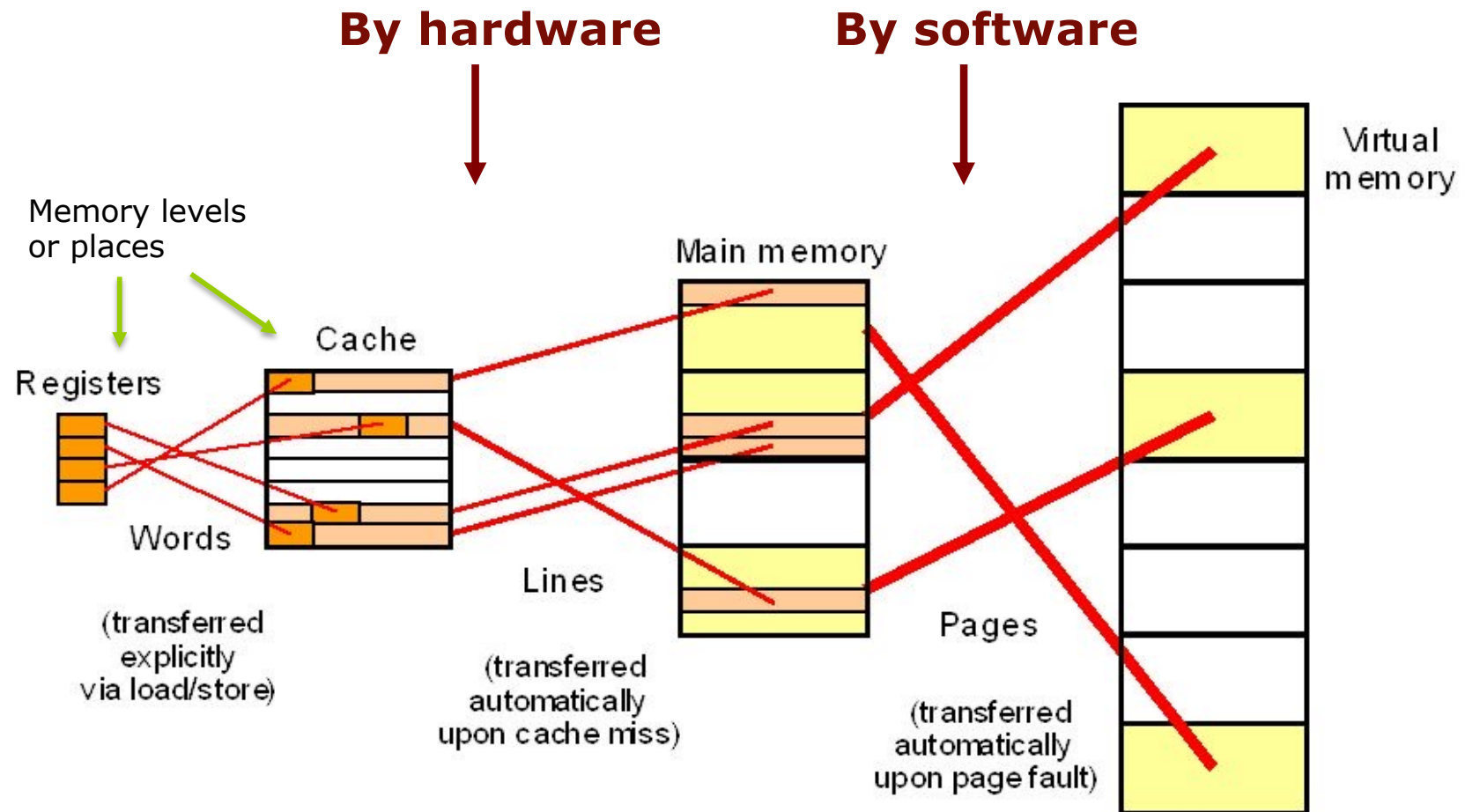
Parallel performance



Parallel Algorithms

- **(recap) Traditional algorithm analysis:** Big O notation
 - Analysis of the number of operations per datum (n)
 - Matriz multiplication: $O(n^3)$ $O(n^{\frac{3}{2}})$ (n=number of elements of C)
 - Sorting (n keys)
 - Bad sorting (burte force): $O(n^2)$
 - Better algorithm: $n \log_2(n)$
 - Best: $([k]n)$
 - Insertion on a data structure (n insertions):
 - Sorted linked list $O(n^2)$
 - Binary tree: $O(n \log_2(n))$
 - Hash table $O(n)$
 - **Impact of data locality?**
 - **Total cycles (#CC) = #CC_{PU}(doing computations) + #CC_{MEM}(waiting for memory)**
 - **What if #CC_{MEM} >> #CC_{PU}**

(recap) Memory hierarchy: the big picture



Data movement in a memory hierarchy.

Parallel Algorithms

Modern

data locality!

~~Traditional~~ **algorithm analysis:** Big O notation

- **(recap) External memory model (simple model)**

- Two memory levels: **external** (e.g., RAM) and **internal** (e.g., cache)
- External memory is organized in Blocks of size B
- Complexity metric used to compare algorithms:
 - Number of blocks transferred between memory levels => **# data moves** (e.g, cache misses)
- Example: sum a vector of elements:
 - Using `Vector<int>` (C++ STL)

Complexity: n/B

($n/4$ if $B=4$, 16 byte cache line)

$v[i]$	$i=0$	$i=1$	$i=2$	$i=3$	$i=4$	$i=5$	$i=6$	$i=7$
Access order, [h]it or [m]iss	1 [m]	2 [h]	3 [h]	4 [h]	5 [m]	6 [h]	7 [h]	8 [h]

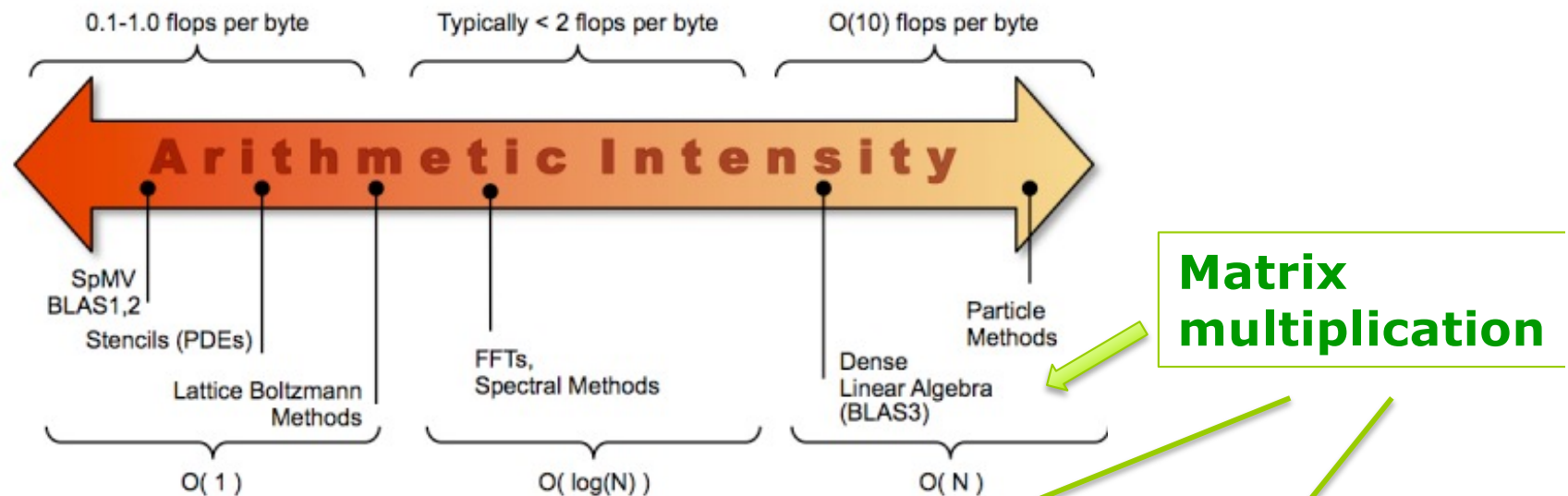
- `ArrayList<Integer>` (Java) ?

- **Our Lecture on matrix multiplications:**

- Count the number of data moves between **memory places** (levels)
- Memory places can be: register, cache, RAM, machines, etc

Parallel Algorithms

- **(recap) Arithmetic Intensity** (operations per ~~byte loaded~~ ^{data moves})



- **Data storage**

- **Regular** : vector, **matrix**
- Irregular (pointer based): tree, graphs

Data accesses
- **Regular** / irregular

[Parallel] Matrix Multiplication

□ What is the best loop ordering for **data locality**?

- 6 possible alternatives (3x2x1 possible loop orders)

```
for ? = 0..N // i, j or k
  for ? = 0..N // ...
    for ? = 0..N // ...
      Cij += Aik x Bkj
```

- How to quantify data locality?

- Where is the hotspot?

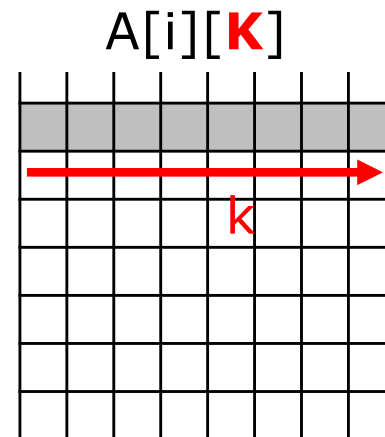
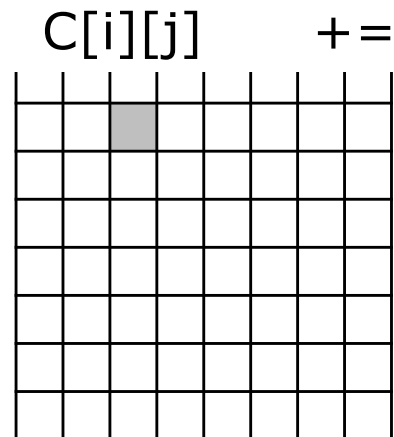
- Analysis under the **external memory model**: **3 options**

1. DOT product

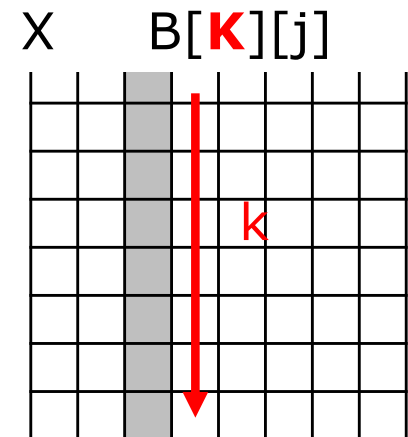
variant: x x K order

```
for ? = 0..N // i or j
  for ? = 0..N // i or j
    for K = 0..N
      Cij += Aik x Bkj
```

ijK and jiK orders have the same **hotspot**: the **for K** loop is executed N^2 times



n/B moves



n moves

C_{ij} and row A_i are read mostly from cache (internal memory)

B_{kj} read from (external) memory

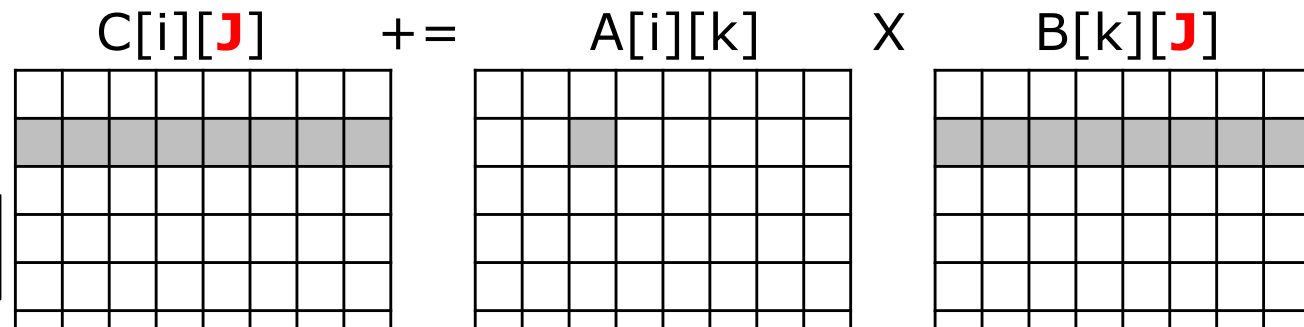
[Parallel] Matrix Multiplication

2. AXPYv1

$$C_{iJ} += a_{ik} \times B_{kJ}$$



Constant inside loop J
(placed in a register)



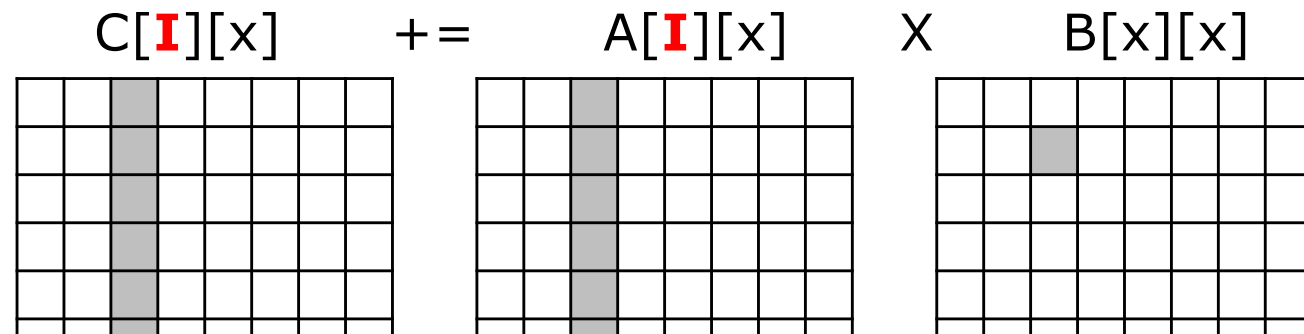
n/B moves

cache (or reg)

n/B moves

x x J order: A_{ik} , Row C_i and row B_k are mostly in cache

3. AXPYv2



n moves

n moves

cache

x x I order: col $C_{?J}$ and col $A_{?K}$ in memory; B_{kj} in cache

[Parallel] Matrix Multiplication

□ What is the best loop ordering for data locality?

■ Summary ($C = A \times B$): (considering the store of C)

- DOT: $x \times K - \pm 2 + N/B$ moves + **N moves** (problem with B)
- AXPYv1: **$x \times J - 2x N/B + 1 + N/B$ moves (ok)**
- AXPYv2: $x \times I - 2xN$ moves + **N moves +1** (problem with C and A)

■ Improve spatial locality

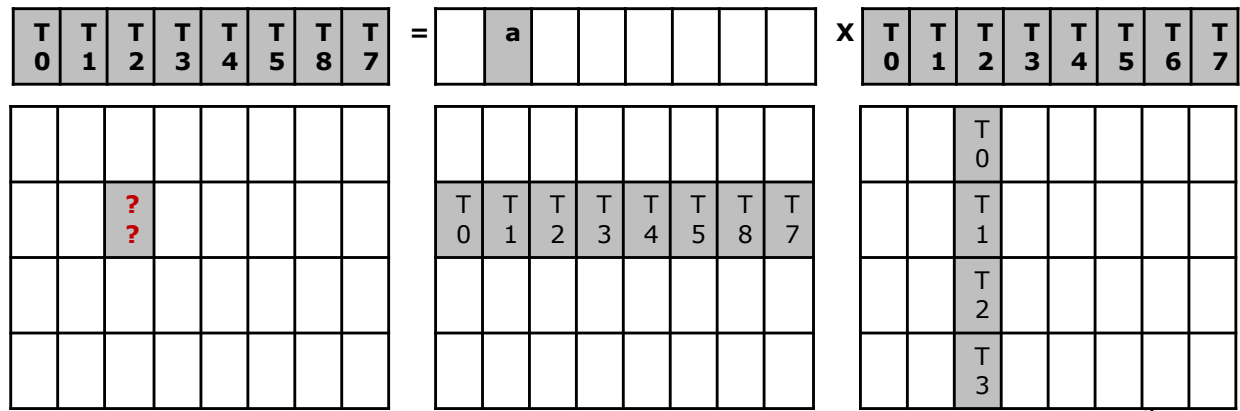
- $x \times K$ – use B^T instead of B
- $x \times I$ – perform C^T and A^T

■ Parallelism: 2 options (AXPY vs DOT)

- Data race on C
 $x \times K$ (DOT)

■ Vectorization

- Parallelism enabled +
- Spatial locality on A and B



[Parallel] Matrix Multiplication

Key insight:

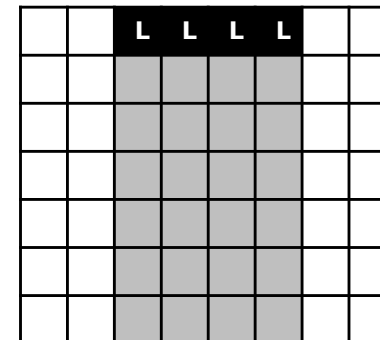
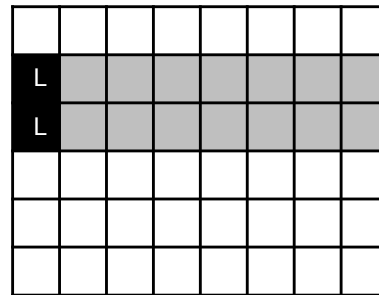
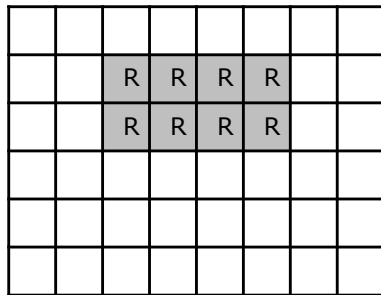
Move data into regs & perform as much computation as possible (limit is number of regs)

□ What about **temporal** data locality?

■ Matrix multiplication by blocks:

- Computes $B_i \times B_j$ elements of C
 - requires B_i elements from A and B_j elements from B
- improves arithmetic intensity
 - e.g.: **1** (1 mul + 1 add)/(2 loads) VS **2 x $B_i \times B_j / (B_i + B_j)$**
 - $K \times x$ - Block of C in internal memory

DOT
with
blocks



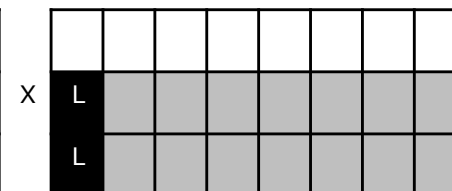
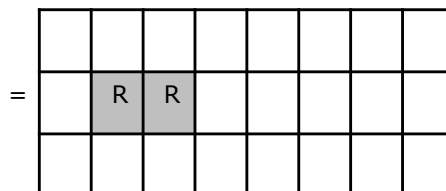
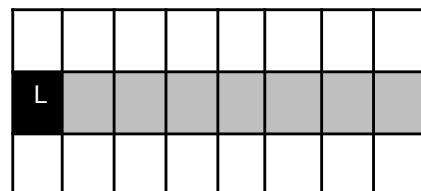
Example in the figure

8 DOTs:

$$\frac{2 * 2 * 4}{2 + 4} = 2,7$$

- Can be applied to all **three** (why three?) versions

AXPY v1
w/blocks
(1x2)



[Parallel] Matrix Multiplication

Multiple levels of blocks

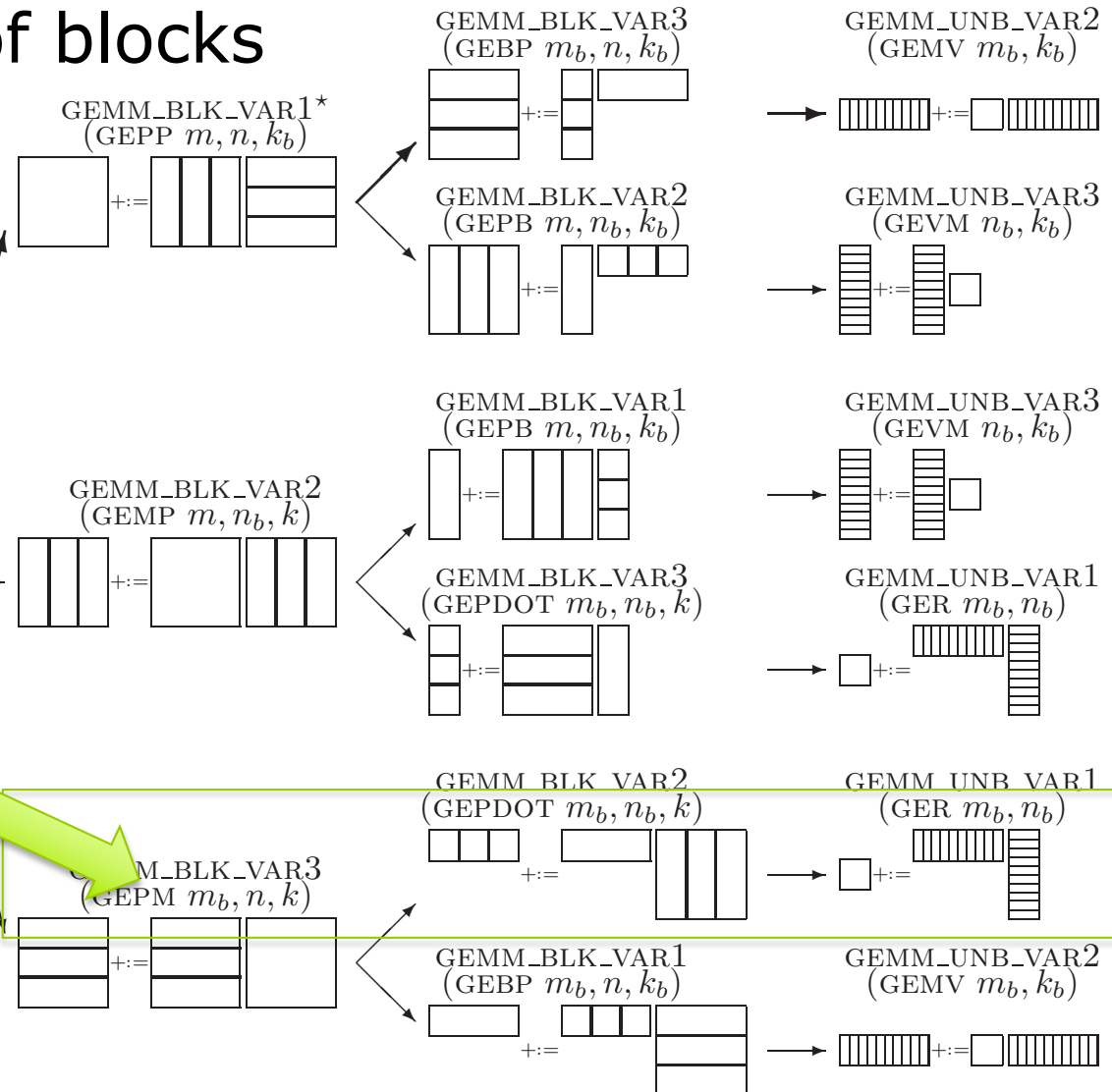
we can apply the same insight at multiple levels:

Move a data block into **cache** & perform as much computation as possible

```

for i = 1 to RA
  for j = 1 to CB
    for k = 1 to RB
      C[i][j] += A[i][k]*B[k][j]
  =>
  for bi = 1 to RA, bi+=blocksize
    for bj = 1 to CB, bj+=blocksize
      for bk = 1 to RB, bk+=blocksize
        for i = bi to bi + blocksize
          for j = bj to bj + blocksize
            for k = bk to bk + blocksize
              C[i][j] += A[i][k]*B[k][j]
  
```

Parallelism???



PL lecture

- Develop an high performance MM in **C**
 - Shared Memory: Use **OpenMP** (also SIMD)
 - Use a Matrix of **doubles, 4096x4096**
 - Measure performance on a **ARM** machine
 - Metric: **execution time**
 - Suggested plan (start with gcc 14.3.0):
 1. **Vectorization** (if necessary use **#pragma omp SIMD**)
 - Compile with specific flags to generate SVE code do A64FX
 - Align data in memory?
(e.g., on gcc use: `double A[4096][4096] __attribute__((aligned (32)))`);
 2. **Register tiling**
 3. **Cache tiling (opt)**
 4. **Parallelism**

PL lecture

□ Support Material – The “best” performance

■ Peak machine performance

Peak FLOPs =

Number of cores × (Task-Level Parallelism)
FLOPS per instruction × (SIMD and FMA)
Instruction per cycle × (Instruction Level Parallelism)
Cycles per second (Frequency)

ARM machine =

48 (cores) x
16 (SVE-512bit) x
2 x
2,0 GHz

around **3000** GFlop/s (peak)

■ Execution time for MM of an NxN matrix (#Flop / Flops)

□ $2 \cdot N^3 / 1400 \times 10^9 = \text{XXX seg}$ (assuming 48 cores & vectorization)

□ Example (512x512, single PU):

▪ $2 \cdot 512 \cdot 512 \cdot 512 / 64 \cdot 10^9 = 0,004 \text{ s}$

(Without vectorization = 0,064s)

■ 10% of this performance should be a good result

□ NOTE: a naïve MM implementation should get around 1% of peak